Cause of Variance using Discrete Cosine Transform Image Compression

This paper investigates variables affecting survivability of images through compression with the Discrete Cosine Transform (DCT). The three main relationships correlated with post-compression quality are smoothness vs rigidity, periodicity vs aperiodicity, and high vs. low contrast. We will visualize our findings, determine the rules that seem to impact post-compression image quality, and introduce the math behind the DCT.

1. Smooth vs Rigid

(a.)

(b.)

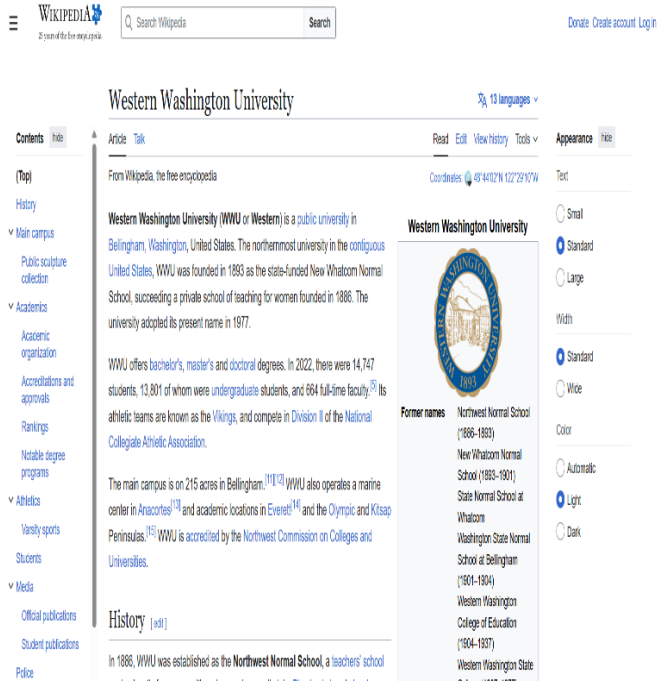
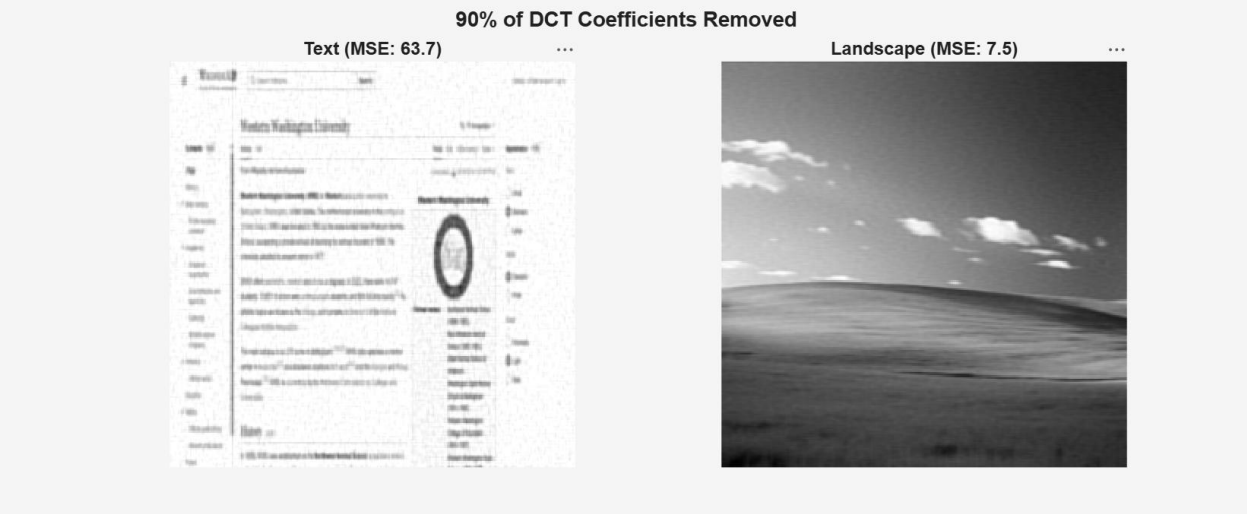


Image a is the Windows XP wallpaper, and image b is Western Washington University's Wikipedia page. Image a is notably smooth, with slight detail in the grass. Image b is very detailed, with rigidness throughout the text, buttons on the screen, and the WWU logo. With 90% compression applied, meaning we remove 90% of the total cosine waves of varying oscillations making up both images, the Wikipedia image does not survive compression in terms of quality.

The amount of change amongst an image's pixels before and after compression can be found using Mean-Squared-Error. Image b looks worse than Image a visually, and this is no placebo: Image b has an MSE of 63.7, 8x larger than Image a's MSE of 7.5.

90% compression using DCT applied to Images (a.) and (b.)



This difference can be explained by observing the spread of frequencies in both images. The Windows XP image consists of smooth frequencies, as well as high and medium-oscillation frequencies with low weights. The Wikipedia image consists of low, medium, and high-oscillation frequencies, with all of them having large weights/coefficients in the DCT C matrix.

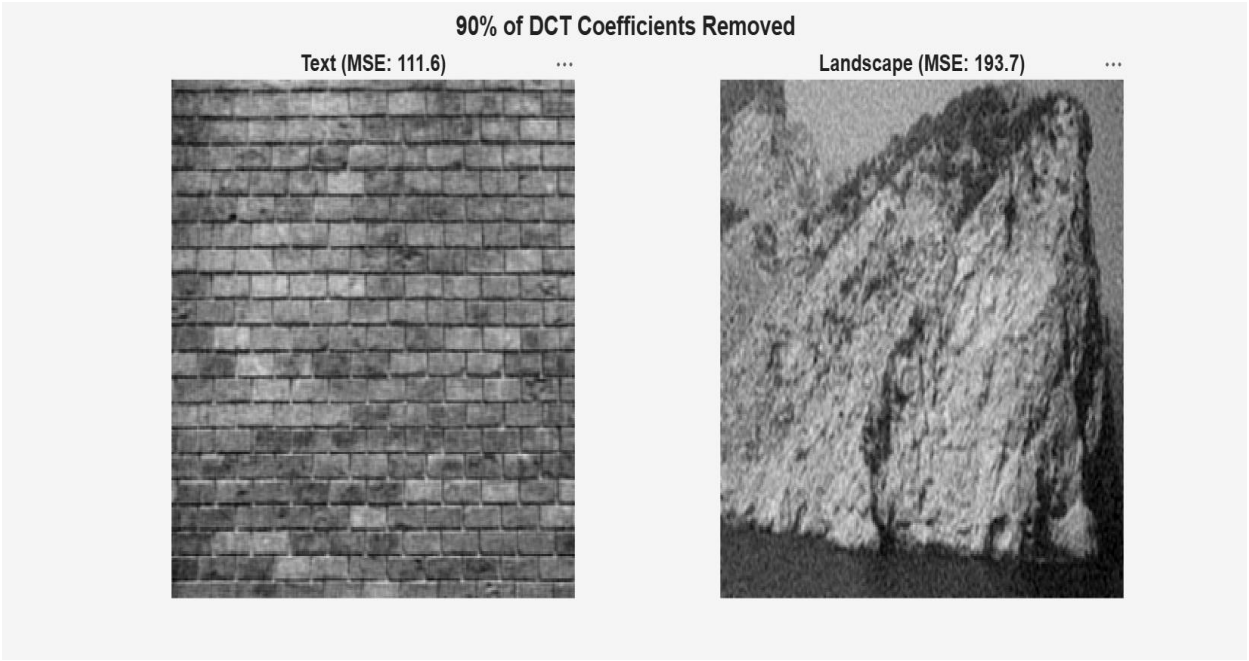
The Wikipedia image uses a large amount of all possible cosine waves, because there are smooth, rigid, and middle-ground elements in the image. With compressing, which is the process of removing 90% of these cosine waves, we end up removing lots of important information encoded by those cosine waves. When 90% compression is applied to the XP image, we remove many high, medium and low-oscillation waves, but most of those were never heavily weighted to begin with, so many of the fundamental cosine waves are still in the image after compression is applied to the entire image.

2. Periodic vs Aperiodic

(a.) (b.)

Image a is a brick wall, and image b is a cliff. (a.) is periodic, as the pattern of one brick repeats throughout the image. (b.) is aperiodic, meaning that there are no repeating patterns in the image. With 90% compression, image b looks worse after compression.

90% compression using DCT applied to Images (a.) and (b.)

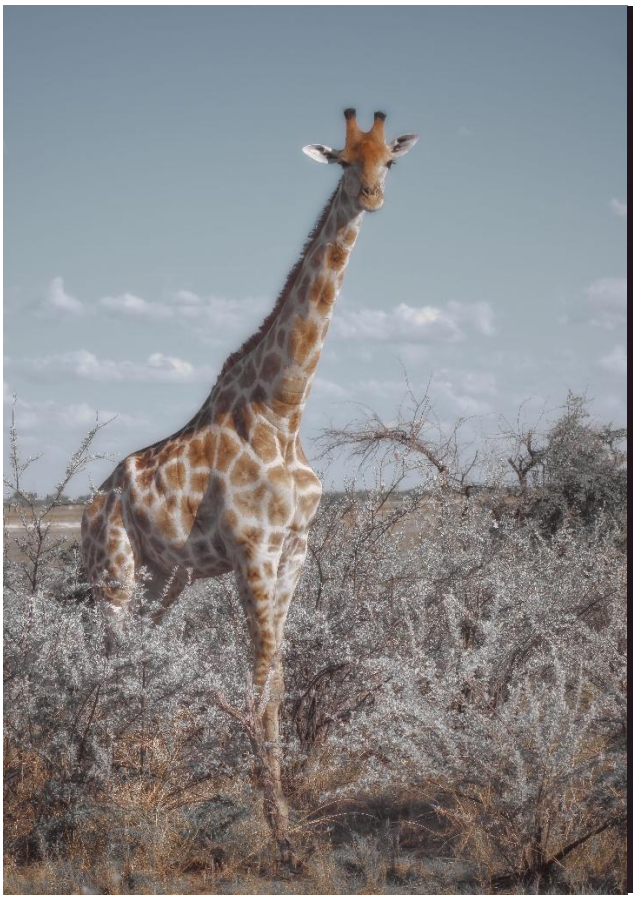


This difference can be explained in energy concentration. Image a reuses the same cosine waves, because the pattern of one brick repeats throughout the image. Image b uses most of all varying-sized oscillation cosine waves.

The brick wall image is periodic, meaning that the same cosine waves will be reused throughout the image, and this results in these cosine waves having large weights. When a percent of all cosine waves are removed via compression, the fundamental waves survive compression, as most of the removed waves never contributed to the image in the first place. The cliff image is aperiodic, meaning that cosine waves are not chosen to be reused, rather the image uses most of all possible cosine waves with varying oscillations.

3. High vs Low Contrast

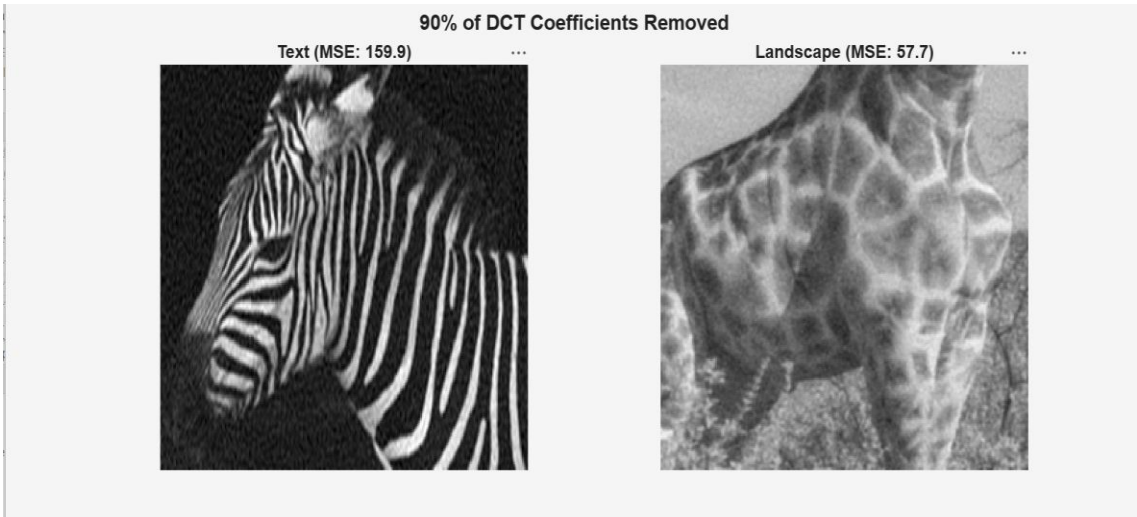
(a.)



(b.)

Image a is a Giraffe, and image b is a Zebra. Image a is low contrast, which means the dark and light pixel values are not far apart. You can tell by looking at some color differences, such as the brown-to-tan color transition in the giraffe's patches. Image b is high contrast, meaning the dark and light values are far apart. You can tell by looking at the zebra's patches, where the color goes from black to white. After compression, the Zebra image looks worse in terms of quality.

90% compression using DCT applied to Images (a.) and (b.)

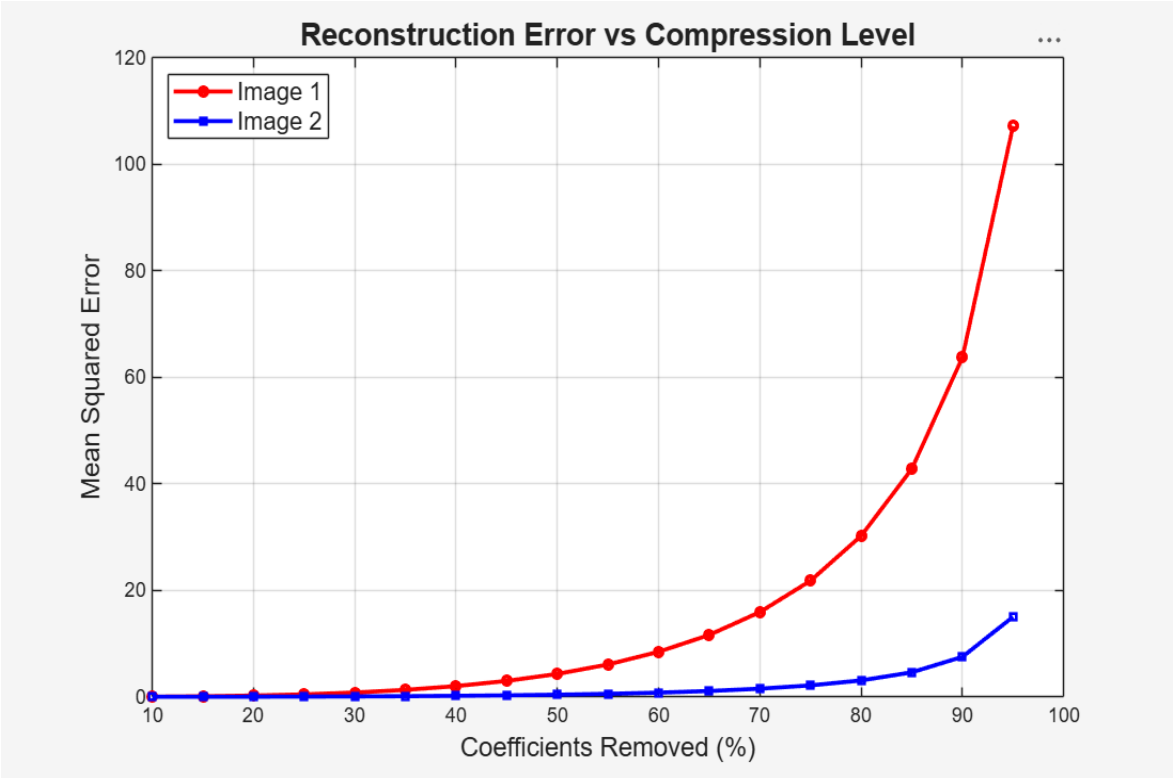


This difference can be explained in the jumps in pixel values. In the zebra image, the black to white transition produces coefficients three to four times larger than the previous coefficients. In the giraffe image, the brown to tan transition has coefficients with a negligible difference. Image compression has a particular feature, where given a certain percentage of compression, you are continuously removing more of the cosine waves with the smallest weights.

In the zebra image, the transition from the dark background to the white on the zebra, back to the black stripes on the zebra creates a range of pixels from around 0-20 to 220-255. The giraffe has pixel values ranging from 100-130 to 150-180 for the transition from tan to brown. This means that the cosine waves in the zebra image have larger weights, because the DCT coefficient is computed as a dot product multiplication between the pixel values and cosine basis vectors. When a percentage of cosine waves are removed in both images, the removed cosine waves in the zebra image had larger weights than the giraffe image, and thus more information was removed from the zebra image.

4. Visual Findings and Summarization

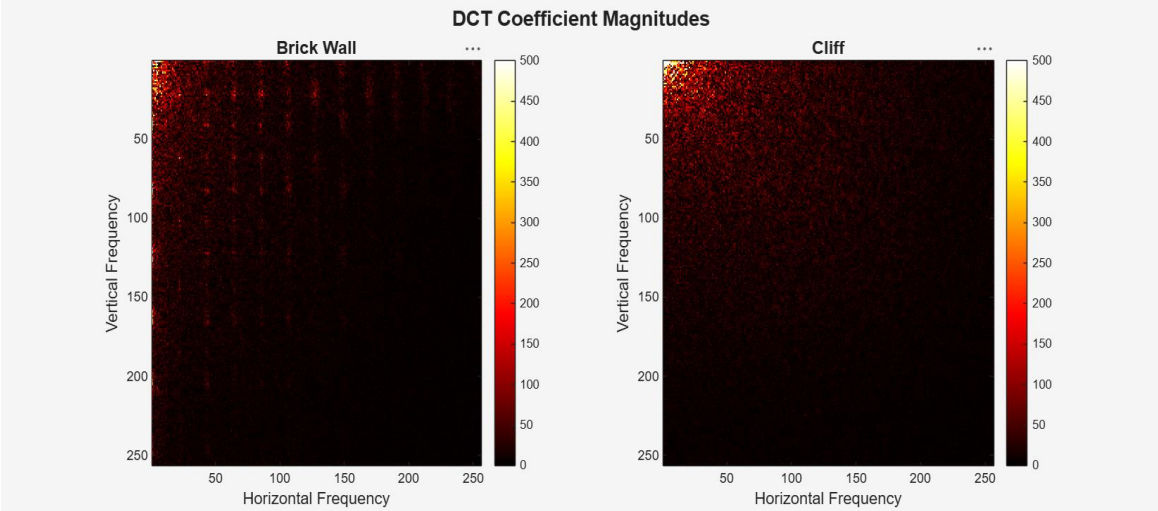
MSE Error of Images (a.) and (b.) as DCT compression grows to 95%



Take the first images shown, which was the Wikipedia page for Western, and the Windows XP wallpaper. After compressing both images while increasing compression rate from 10% to 95%, the MSE of both images diverge.

This graph, and the previous three image comparisons, display the compression effect of images based on their unique cosine wave distribution. Images with a selective usage of cosine waves can survive compression, whereas images with a large usage of most cosine waves will fail compression. We saw images like the XP wallpaper and the brick wall be selective with what cosine waves they used. We saw the giraffe image use minimal gaps in color transitions or pixel values, whereas the zebra image used larger weights as a product of large color differences and their transitions. Being selective in what cosine waves are used is how an image survives compression

Heat map identifying the repeating brick pattern present in the brick image.



This is a heatmap, which is a grid of numbers displayed as colors. Each cell in the grid a DCT coefficient computed for one cosine wave. The brightness at a particular point in the graph represents the strength of the cosine wave at that point. The brick wall heatmap has multiple bright lines starting from the top and moving downwards, which represent the repeated pattern of a brick being used in the image, which is the same as copies of the same cosine waves being reused, or these cosine waves having a larger weight.

The cliff heatmap has no lines, because no pattern repeats, meaning no cosine wave of a particular oscillation is reused.

5. DCT Algorithm

DCT formula to produce basis cosine waves.

$$C(i,j) = \frac{\sqrt{2}}{\sqrt{n}} \cdot a_i \cdot \cos\left(\frac{i \cdot (2j + 1) \cdot \pi}{2n} \right)$$

This formula produces the entries of the DCT matrix C. The process of DCT compression beings by carrying out this formula to produce n cosine waves for any input of n elements. The matrix multiplication of C and input x yields weight vector y, where each element represent how much a cosine wave contributes to input x.

Determining how much each cosine wave contributes to our input

Finding weights for our input

0.5	0.5	0.5	0.5
0.6533	0.2706	-0.2706	-0.65330
0.5	-0.5	-0.5	0.5
0.2706	-0.6533	0.6533	-0.2706

1
0
-1
0

=

0	0.9239	1.0	-0.3827

C(i,j) = (sqrt(2)/sqrt(n)) * a_i * (cos(i * (2j+1) * 3.14 / (2n))

Say our input file is x = [1,0,-1,0].

x = [1, 0, -1, 0] is stored as w = [0, 0.9239, 1.0, -0.3827] in the machine. Cosine wave 4 is smallest and is least-contributing to our file.

From here, we can see that cosine wave 1 contributes nothing (0), cosine wave contributes a magnitude of 0.9239, cosine wave 3 has 1.0, and cosine wave 4 has -0.3827. We can see that cosine wave 4 contributes the least when comparing all nonzero weights. If we choose to compress our image, the image compression algorithm will delete the least-contributing cosine vector for every 25% of compression applied. We can simply turn the fourth weight to zero, then do row-wise multiplication using C and w to return a compressed version of x.

Compressing our file by 25%, by removing 1 cosine vector (1/4) which contributed the least to the input.

Removing the smallest weight

0.5	0.5	0.5	0.5
0.6533	0.2706	-0.2706	-0.65330
0.5	-0.5	-0.5	0.5
0.2706	-0.6533	0.6533	-0.2706

•

0
0.9239
1.0
0

=

1.1036	-0.2500	-0.7500	-0.1036

x = [1, 0, -1, 0] becomes x = [1.1036, -0.2500, -0.7500, -0.1036]

This input file is close to our original input x = [1,0,-1,0]

And our machine stores 3 weights instead of 4, reducing file size by 25%

MATLAB code to run DCT on an input array x = [1, 0, -1,0], and reconstruct the output.

```
%% Creating a 4x4 DCT Matrix
n = 4;
C = zeros(n, n);
for i = 0:n-1
    for j = 0:n-1
        if i == 0
            a_i = 1/sqrt(2);
        else
            a_i = 1;
        end
        C(i+1, j+1) = (sqrt(2)/sqrt(n)) * a_i * cos(i*(2*j+1)*pi/(2*n));
    end
end
```

```
%% Encoding
x = [1; 0; -1; 0];
weights = C * x;
```

```
%% Removing the smallest weight and decoding
for step = 1:4
    result = zeros(1, n);
    for i = 1:n
        result = result + weights(i) * C(i, :);
    end
    fprintf('x = [%s]\n', num2str(round(result, 4)));
    magnitudes = abs(weights);
    magnitudes(weights == 0) = Inf;
    [~, smallest] = min(magnitudes);
    weights(smallest) = 0;
end
```

The purpose of DCT is that it allows us to compress an R^k dimensional input by any amount, while reconstructing an output similar to our original input x by an error amount for each input x_i to x_n.

5.1 Orthogonality of C

One notable characteristic of the C matrix containing the cosine waves, is that C is an orthogonal matrix. The process of decoding the weight vector w to reconstruct a compressed output used row-wise multiplication with C and w, which is identical to doing C^T * w. Since C is orthogonal, C^T * C = I. After compute weights using w = Cx, C being orthogonal means we can recover x exactly using C^T * w. Since x is recoverable with no loss of information, the algorithm depends on us to set weights in w to zero to compress the input through the transformation.

5.2 Extension to R^2

Images are represented as n x m matrices. The weight vector w will instead be computed using the formula W = C * X * C^T, instead of W = C * X in R^1. This produces an n x m matrix of weights. In R^1, cosine waves are summed to build the reconstructed input, and these waves varied throughout one direction (the cosine waves varied from element 1 to element n). In R^2, the cosine waves vary horizontally vertically, and thus each weight in w must describe how much a cosine wave contributes to an input in both directions. In a n x m weight matrix W, a value W(i,j) represents how much one cosine wave contributes to the image in the vertical and horizontal direction of the frequencies of the image.

6. Conclusion

This analysis found three properties of an image that affect how well it survives DCT compression, that being smoothness, periodicity, and contrast. Smooth images, like the XP wallpaper, concentrate their information into select cosine waves, therefore surviving compression, or mathematically the removal of 90% of all cosine waves that contribute to the image. The brick wall image repeated the same pattern throughout, and the giraffe image was inherently conservative in that it had a smaller range of pixel values and thus smaller weights. The three properties reflect being selective and repetitive in cosine wave usage, and thus selective in what data is in our input. We could visualize these properties changing the measured difference (MSE) in pixels before and after compression, and the heatmap revealed concentrated energy at specific cosine wave frequencies. These results suggest that the survivability of an image through DCT compression is determined by how much of the image can be represented by a small number of cosine waves.

Written by William Rongerude